COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	07/29/2026	**DATE SUBMITTED:**	07/29/2026
NAME:	Juan Paulo C. Lara	**INSTRUCTOR:**	Engr. Jimlord Quejado

Seatwork NO. 4.1 **Array-Based vs. STL Stack Implementations**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Implement a stack using the C++ STL (std::stack).

```cpp
#include <iostream>
#include <stack>

int main(){

    std::stack<int> stack1;
    std::stack<int> stack2;

    std::cout << "Testing the stack STL" << std::endl;

    // isEmpty
    std::cout << "Is the stack empty? " << stack1.empty() << std::endl;

    // push
    stack1.push(10);
    std::cout << "The top of the stack is " << stack1.top() << std::endl;
    stack1.push(9);
    std::cout << "The top of the stack is " << stack1.top() << std::endl;
    stack1.push(8);
    std::cout << "The top of the stack is " << stack1.top() << std::endl;
    stack1.push(7);
    std::cout << "The top of the stack is " << stack1.top() << std::endl;

    // pop
    stack1.pop();
    std::cout << "The top of the stack is " << stack1.top() << std::endl;
    std::cout << "Is the stack empty? " << stack1.empty() << std::endl;
    std::cout << "THe size of the stack is " << stack1.size() << std::endl;

    // emplace
    stack1.emplace(11);
    std::cout << "The top of the stack is " << stack1.top() << std::endl;
    std::cout << "THe size of the stack is " << stack1.size() << std::endl;

    // swap
    stack2.push(12);
    stack1.swap(stack2);
    std::cout << "The top of the stack1 is " << stack1.top() << std::endl;
    std::cout << "The top of the stack2 is " << stack2.top() << std::endl;

    return 0;
}
```

Figure 1. Code Structure of ILO A: C++ STL Stacks.

This code contains the basic ways of manipulating stacks in the C++ language using the stack library, which contains a simple template of the implementation of the core function of pushing and popping stacks, and also other functions like the emplace, empty, and swap functions. These functions from the STL library serve as the standard way of implementing stack management, but it is not customizable by the coder other than the functions provided by the library. In terms of how this code works, it first declares the variables for stack1 and stack2, which is used later for swapping. The first values (10, 9, 8, 7) are pushed to the STL stack using the push function with the parameter containing the four respective values. The pop function automatically takes out the latest element of the stack and reduces the number of the stack as well. When the number of the stacks is displayed, the number is subtracted from the number of the total stacks before. Moving over to the swap, this process involves pushing one value to another stack, so it can be used as the value for swapping. Then, it executes a swap function swapping the value of stack2 from stack1.

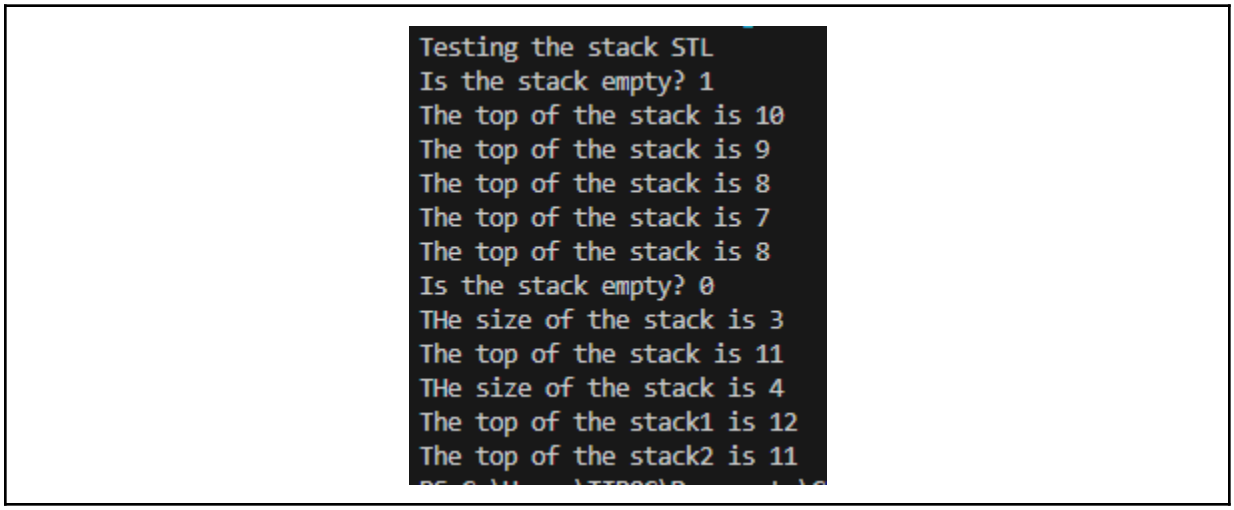


Figure 2. Code Output of ILO A: C++ STL Stacks.

This figure shows the code output containing the multiple stack operations using the STL stack library and its functions. The part of the code checks if it is empty before the values of the stack are entered, added to the stack, and displayed in the console. The outputs containing the first five instances of “The top of the stack: contain the first five values entered in the driver code to be added to the stack. After this, a check is displayed whether the stack is empty, which is not anymore. Then, the size of the stack is displayed after a pop function is executed. The same process is done for the emplace and the swap values. For the swap values, the only change in the output is the exchange of values from stack1 and stack2.

ILO B.1: Implement a stack using a Static Array.

```cpp
#include <iostream>

// global declaration
#define maxCap 100
int stackArr[maxCap];
int top = -1, newData;

// prototype function
void push();
void pop();
void Top();
bool isEmpty();
bool isFull();


int main(){
    // main driver code
    int choice;

    while (true){
        std::cout << "============================================" << std::endl;
        std::cout << "Stack Operations\n";
        std::cout << "1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull" << std::endl;
        std::cout << "============================================" << std::endl;
        std::cin >> choice;

        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout << "Is the stack empty? " << isEmpty() << std::endl;
            break;
            case 5: std::cout << "Is the stack full? " << isFull() << std::endl;
            break;
            default: std::cout << "Invalid choice" << std::endl;
            break;
        }
    }

}

```

```cpp
// function definition
bool isEmpty(){
    // check the top if it is -1
    if(top == -1) return true;
    return false;
}

bool isFull(){
    // how do we verify if the stack is full
    if(top == maxCap - 1) return true;
    return false;
}

void push(){
    // error checking
    if(isFull()){
        std::cout << "Stack Overflow" << std::endl;
        return;
    }

    // pushing to the stack
    std::cout << "Enter a new value" << std::endl;
    std::cin >> newData;

    // how do we insert the data into the stack?
    stackArr[++top] = newData;
}

void pop(){
    // errpr checking
    if(isEmpty()){
        std::cout << "Stack Underflow" << std::endl;
        return;
    }

    // DIsplay the value that we are going to pop
    std::cout << "Popping: " << stackArr[top] << std::endl;

    // decrement the value from the stack
    top--;

}
```

```
87
88  void Top(){
89      // error catching
90      if(isEmpty()){
91          std::cout << "The stack is empty" << std::endl;
92          return;
93      }
94
95      // check the top element
96      std::cout << "Top Element: " << stackArr[top] << std::endl;
97
98  }
99
```

Figure 1. Screenshot of the Code Implementation of Stacks using Arrays.

This code screenshot contains the code for the implementation of the stack using arrays, including the standard structure of a C++ program, which consists of a global declaration, prototype function, function definition, and driver code. The main driver code contains the user input menu for using this program's features, which are isEmpty, isFull, top, push, and pull. These functions perform the basic operations done inside a stack, including error catching to detect a stack overflow and underflow. Most of the code contains conditional if statements to check for stack errors to keep the program from crashing with a runtime error. Conditional statements are also used in determining if the stack is full or empty. Using arrays requires more preparation than the STL method, as it requires declaring an array, setting a variable to store the maximum number of elements, and the newData for user input.

```
==============================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull
==============================================
4
Is the stack empty? 1
==============================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull
==============================================
1
Enter a new value
2
==============================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull
==============================================
3
Top Element: 2
==============================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull
==============================================
2
Popping: 2
==============================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull
==============================================
4
Is the stack empty? 1
==============================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull
==============================================
1
Enter a new value
3
==============================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull
==============================================
5
Is the stack full? 0
```

Figure 2. Code Output of the Program Implementation.

This screenshot contains the console output of the program using arrays to manage stacks. There are five user input choices in the program, which are push, pop, top, isEmpty, and isFull. The push and pop options allow user

input to input their desired value into the console with integer numbers, which then goes to the functions that perform the push and pop operations in the stack. The isEmpty and isFull are options that check if the stack is empty or full, and print the message with the boolean statement determining the state of the stack. The top option is where the program calls the top function to print the value of the top or latest part of the stack.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Concept clarification of stacks**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to explain the concepts of stacks**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.